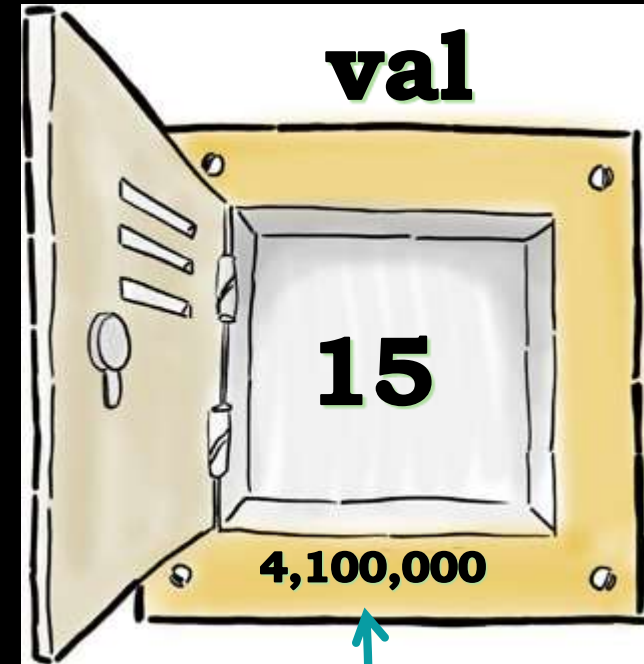


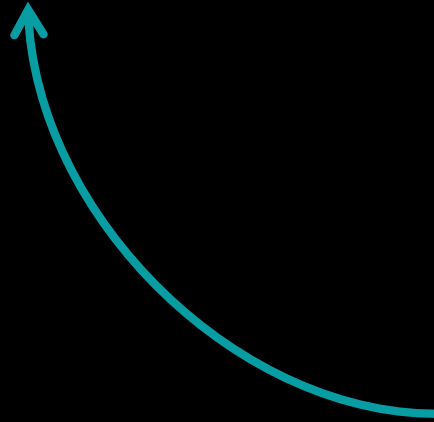
pointers

```
int val = 15;
```



The latitude
variable is at
memory location
4,100,000.

```
int *address_of_val = &val;
```

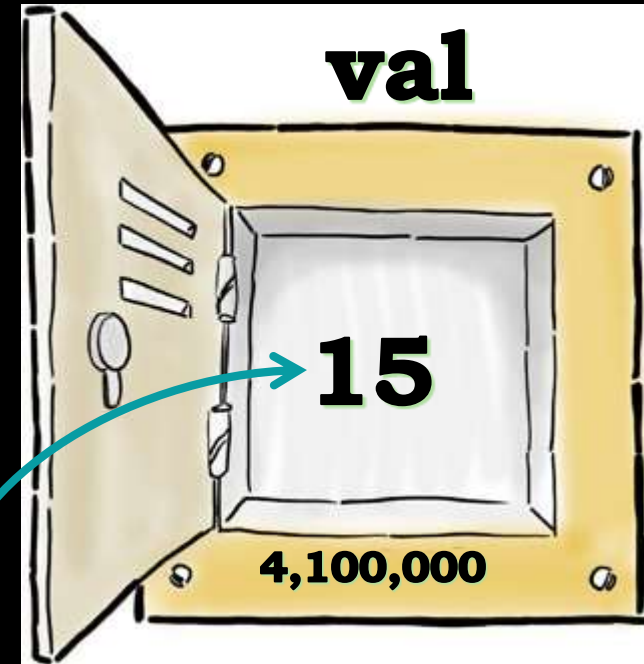


This is a pointer
variable for an address
that stores an int.

Read the contents of an address

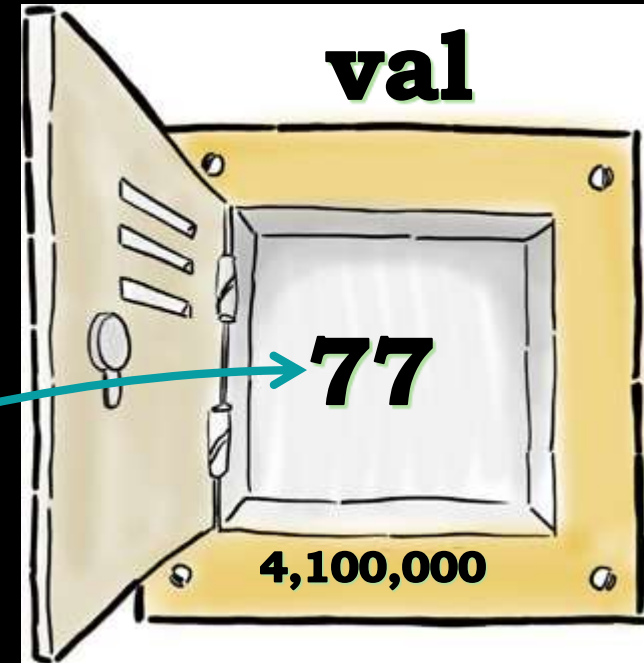
```
int value_stored = *address_of_val;
```

This will read the contents at
the memory address given by
address_of_val.
This will be set to 15



Change the contents of an address

```
*address_of_val = 77;
```



This will change the contents
of the original x variable to 77

Val and val2 have the same
address and value.

```
int &val2 = val ;
```



```
5 ▾ int main() {  
6  
7   int x = 15;  
8   cout << &x<<" ## "<< x;  
9   cout<< endl;  
10  
11  int &y = x;  
12  cout << &y<<" ## "<< y;  
13  
14  
15      return 0;  
16 }
```

Output

```
0x7ffcdea38dc4 ## 15  
0x7ffcdea38dc4 ## 15
```

```
5 int main() {  
6   int arr[5] = {1, 3, 8, 9, 5};  
7   int x = arr[3];  
8   cout << x<< endl;  
9   arr[3] = 25;  
10  cout << x << endl;
```

Output
9
9

```
5 int main() {  
6   int arr[5] = {1, 3, 8, 9, 5};  
7  
8   int &x = arr[3];  
9   cout << x<< endl;  
10  
11   arr[3] = 25;  
12   cout << x << endl;  
13  
14   for(auto &i : arr)  
15       cout << i << " ";
```

Output

9

25

1 3 8 25 5

```
5 int main() {  
6     int x = 25;  
7     int *ptr = &x;  
8  
9     cout << *ptr << " ## " << ptr << " ## " << &ptr;  
10    cout << endl;  
11    // cout << &x;  
12
```

Output

25 ## 0x7ffdbbe7686c ## 0x7ffdbbe76860

```
5 ▾ int main() {  
6   int x = 25;  
7   int *ptr = &x;  
8  
9   cout << *ptr <<" ## "<< ptr << " ## " << &ptr;  
10  cout <<endl;  
11  cout << &x;
```

Output

```
25 ## 0x7ffc9033843c ## 0x7ffc90338430  
0x7ffc9033843c
```

Pointer to an Array

```
4 int main() {
5     int arr[] { 3, 5, 7, 9 };
6
7     // value of array is address: to first element
8     cout << arr << "\n";    // 0x7f
9
10    int &val = arr[0];
11
12    cout << val << " " << &val << "\n"; // 3 0x7f
13    cout << &arr[0] << "\n";    // 0x7f
14
15    int *ptr = arr;
16    // also value of ptr is address
17    // 3 0x7f 0x2aa
18    cout << *ptr << " " << ptr << " " << &ptr << "\n";
19
20    // point to array: you can use it as same as array
21    ptr[0] = 10, ptr[1] = 20;
22
23    for (auto &val : arr)
24        cout << val << " ";
25    cout << "\n";    // 10 20 7 9
26}
```

- Data Structure:

- A particular way of storing and organizing data in a computer so that it can be used efficiently
- It is a group of data elements stored together under one name
- It is also referred as “Abstract Data Type (ADT)”

- For what purpose?

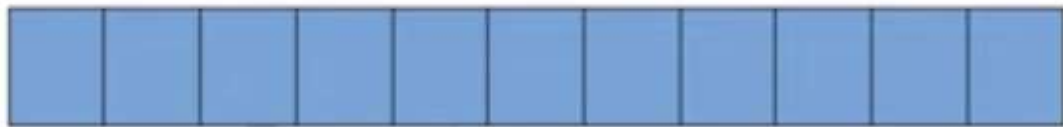
- To facilitate efficient
 - storage of data
 - retrieval of data
 - manipulation of data

- Design Issue:

- The challenge is to select the most appropriate data structure for the problem

List of Data Structures

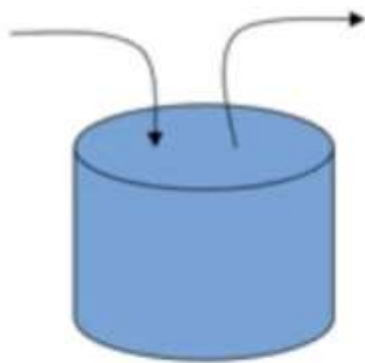
- Arrays



- Linked List



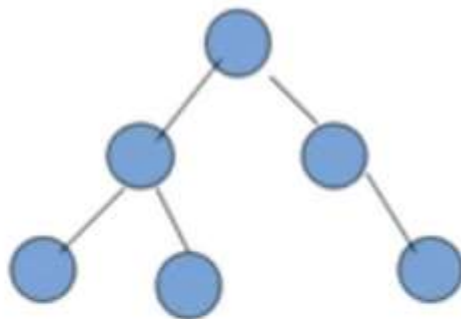
- Stacks



- Queues



- Trees

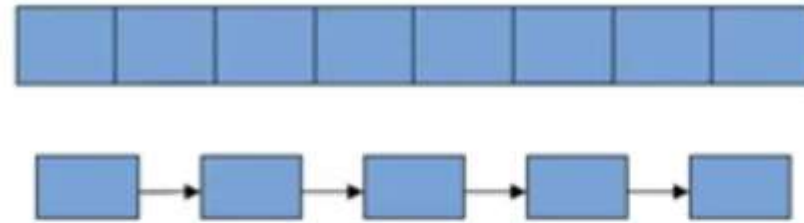


Classification of Data Structures

- Based on Existence:

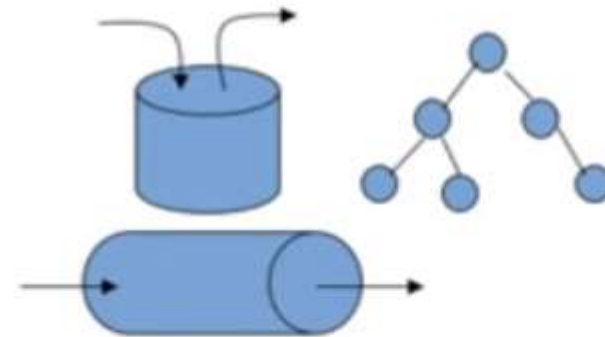
- **Physical** data structures

- Can be created independently
 - Array
 - Linked List



- **Logical** data structures

- Can't be created independently
 - All others e.g.
 - stack, queues, trees, graphs

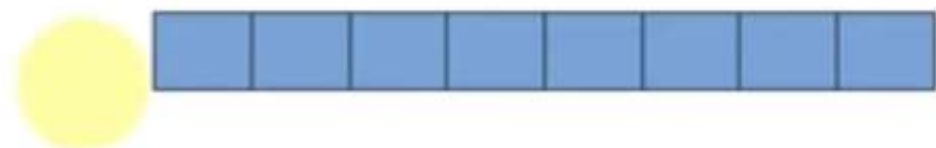


Classification of Data Structures

- Based on Memory Allocation:

- **Static** (or fixed sized) data structures

- Such as arrays



- **Dynamic** data structures (change size as needed)

- Such as Linked Lists

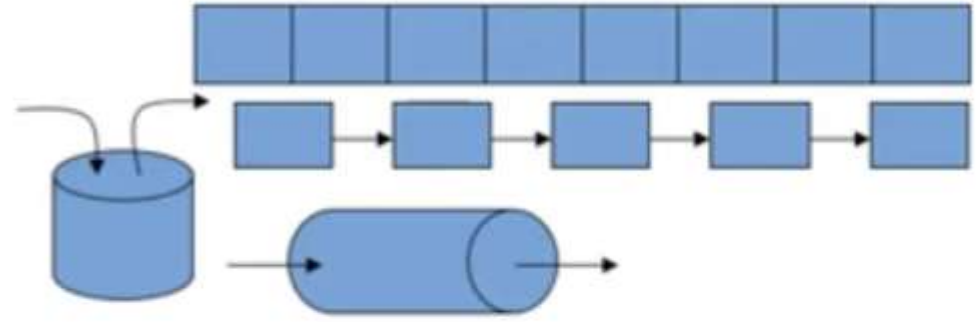


Classification of Data Structures

- Based on Representation

- **Linear** data structures

- Arrays
 - Linked lists
 - Stack
 - Queues



- **Non-linear** data structures

- Trees
 - Graphs



Operations on Data Structures

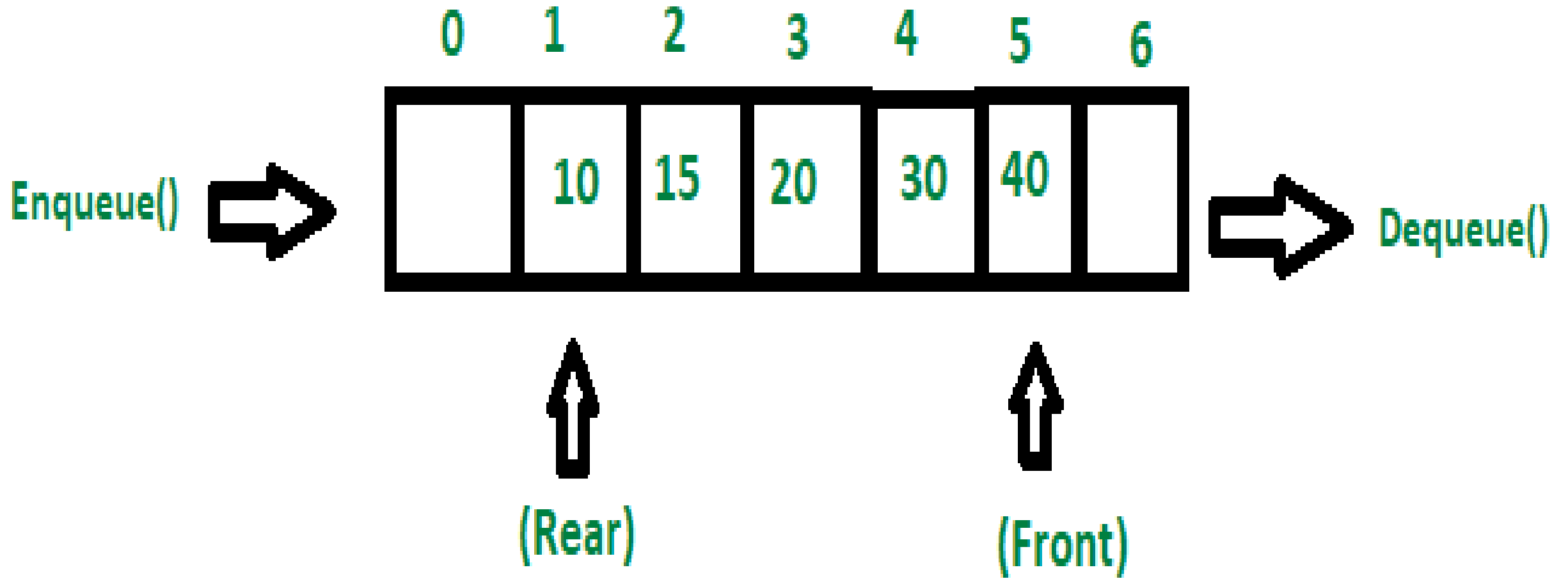
- Traversing
 - Accessing/visiting each data element exactly once so that certain items in the data may be processed
- Searching
 - Finding the location of a given data element (key) in the structure
- Insertion:
 - Adding a new data element to the structure

Operations on Data Structures

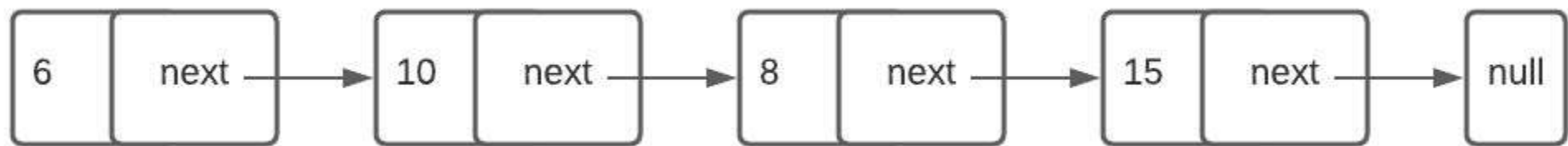
- Deletion
 - Removing a data element from the structure
- Sorting
 - Arrange the data elements in some logical fashion
 - ascending or descending
- Merging
 - Combining data elements from two or more data structures into one

Queue Data Structure

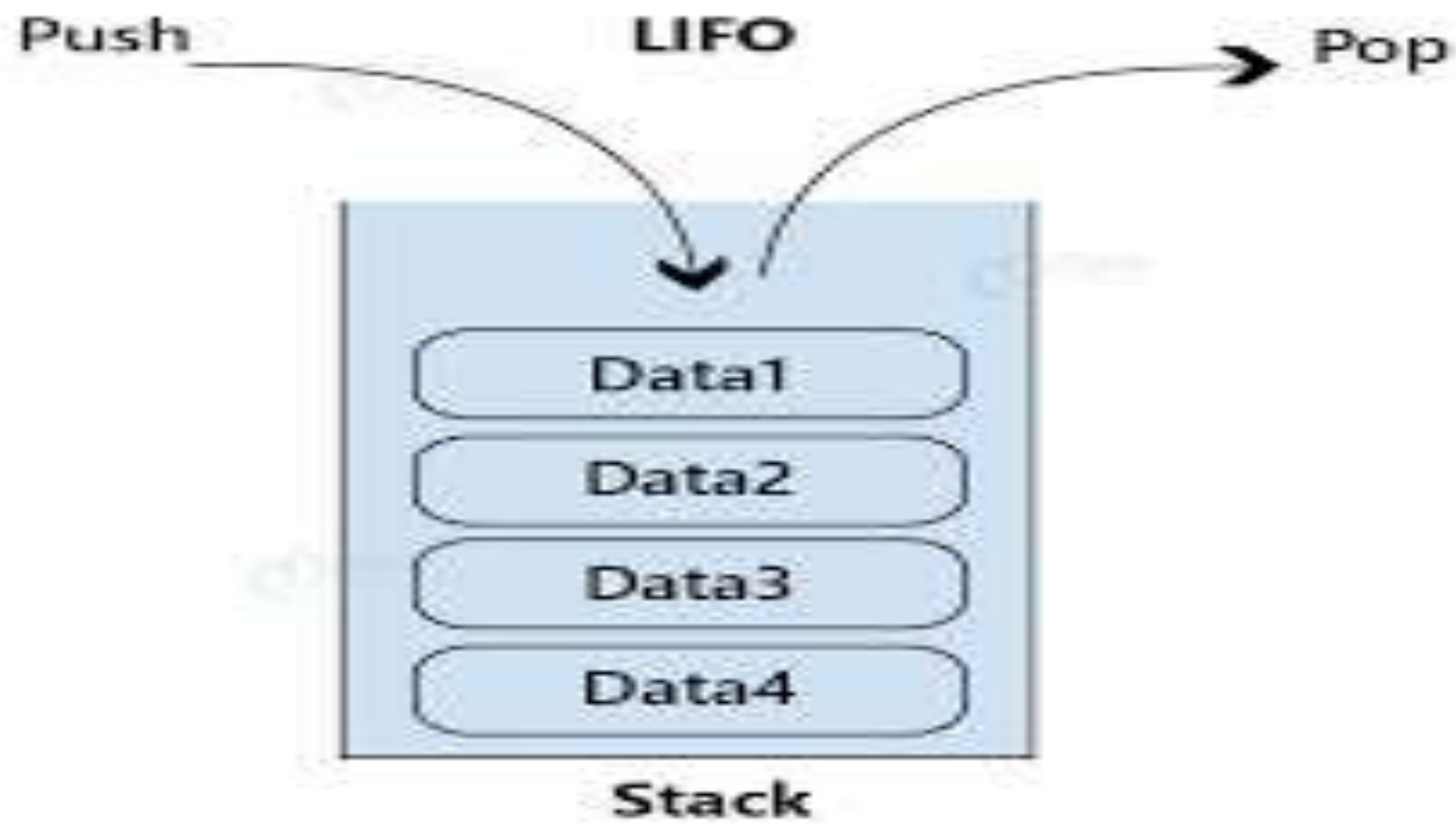
(FIFO).



Singly Linked List



Stack Data Structure



Singly Linked List

- **We can create a single integer: `int* val = new int`**
- **We can create several separate integers the same way**
- **We can expand with more separate values**
- **But this is not useful so far!**
- **We need them to be linked not separate!**

```
int *val1 = new int ;  
int *val2 = new int ;  
int *val3 = new int ;  
int *val4 = new int ;
```

Can we group them together?

What if we also create other 4 pointers

- **to link them together?**
 - **Link from 1 to 2**
 - **Link from 2 to 3**
 - **Link from 3 to 4**
 - **Link from 4 to ??? Flag to stop!**

Node Data Structure

What about a class that has 2 variables

- **The int value**
- **The pointer to the link to the next value?!**
- **Let's call it a **Node****

```
struct Node{  
    int data ;  
    Node* next;  
    Node(int data) : data(data) {}  
};
```

Let's create 4 objects and set data

- **Let's link each object with next one**
- **To mark the last node**
 - Use **nullptr**

```
Node* node1 = new Node(6);  
Node* node2 = new Node(12);  
Node* node3 = new Node(25);  
Node* node4 = new Node(10);
```

```
node1 -> next = node2;  
node2 -> next = node3;  
node3 -> next = node4;  
node4 -> next = nullptr;
```

Now, given ONLY the first node (head), we can move to any next node

```
cout << node1->next->next->next->data << "\\n";  
cout << node2->next->next->data << "\\n";  
cout << node3->next->data << "\\n";  
cout << node4->data << "\\n";
```

Printing Nodes Chain

```
void print1(Node* head)
{
    while(head != nullptr)
    {
        cout<< head->data<<" ";
        head = head->next;
    }
    cout<<"\n";
}
```

task1 Rewrite the print1 function using for loop instead of while loop

Printing Nodes Chain: Recursively

```
void print2(Node* head)
{
    if (head == nullptr)
    {
        cout<< "\n";
        return;
    }
    cout << head->data<<" ";
    print2(head->next);
}
```

Printing Nodes Chain reversed : Recursively

```
void print_reversed(Node* head)
```

```
{
```

```
    if (head == nullptr)
```

```
        return;
```

```
    print_reversed(head->next);
```

```
    cout << head->data<<" ";
```

```
}
```

Singly Linked List

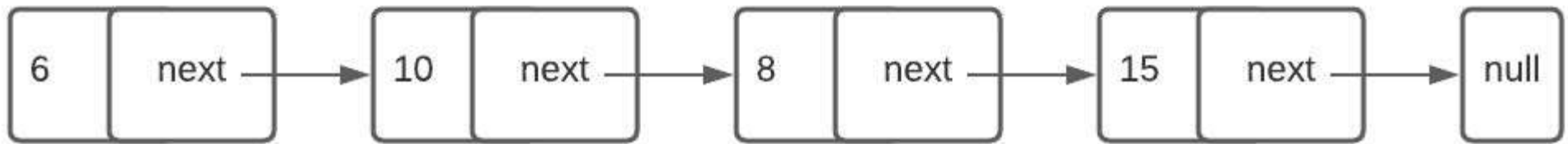
- **Linked List is a sequence of nodes, where each node contains data and link to the next node creating a dynamic connected chain of values.**
- **The first node is called head and the last one is called tail**

Singly Linked List: Data Structure

```
struct Node{  
    int data;  
    Node* next;  
    Node(int data) : data(data){}  
};
```

```
class LinkedList{  
    private:  
        Node *head{};  
        Node *tail{};  
    public:  
        void print();  
        void inser_end(int value);  
        ...
```

Singly Linked List: insert_end



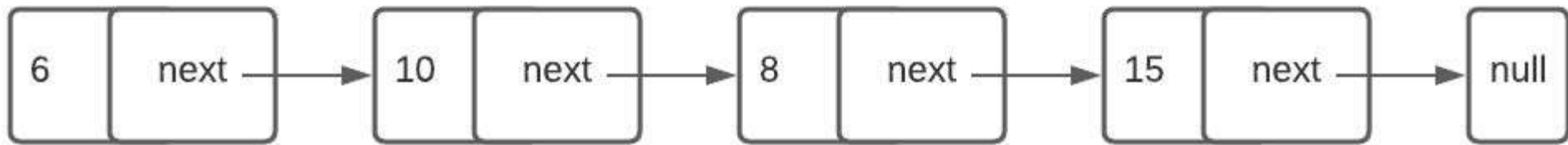
```
void insert_end(int value)
{
    Node* item = new Node(value);
    if (!head)
        head = tail = item;
    else{
        tail->next= item;
        tail = item ;
    }
}
```

Singly Linked List: delete_front

```
void delete_front() {  
    assert(length);  
    Node* cur = head->next;  
    delete_node(head);  
    head = cur;  
    if(length <= 1)  
        tail = head;  
}
```

task2 Get_nth item:

- Let's implement: **Node* get_nth(int n)**
- The function get the nth node. If not found, it returns NULL
- Below: **get_nth(4)** return Node with value **15**



task3 Merge Two Sorted Lists

- You are given the `heads` of two `sorted` linked lists `list1` and `list2`.
- `Merge` the two lists into `one sorted list`. The list should be made by splicing together the nodes of the first two lists.
- Return *the head of the merged linked list*.
- Ex:
- **Input:** `list1 = [1,2,4]`, `list2 = [1,3,4]` **Output:** `[1,1,2,3,4,4]`

task4:Reverse linked list

- Given the head of a singly linked list, reverse the list, and return the reversed list.
- Ex:
- **Input:** head = [1,2,3,4,5] **Output:** [5,4,3,2,1]